



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

Sistemas Distribuidos

REPORTE LABORATORIO 8

GRUPO: 02

ALUMNOS:

MEDRANO SOLANO ENRIQUE - 320152841

FECHA DE ENTREGA: 17 de octubre de 2026

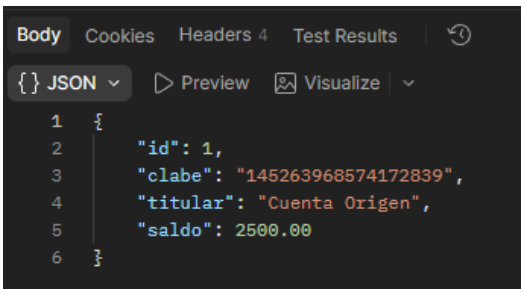
PROFESOR: Ing. Oscar Manuel Ruiz Hurtado

Desarrollo.

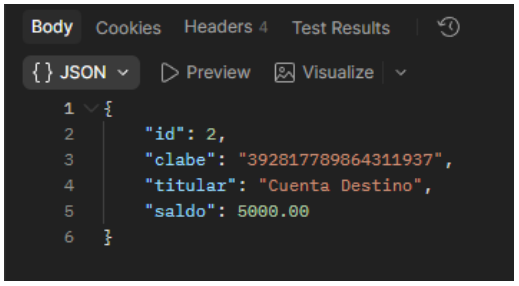
1. El problema.

Los saldos antes y después de la Parte 0, y en una frase por qué pasó.

Antes de verificar los saldos de la cuenta, debemos de generar las cuentas CLABE, por tanto se realiza los siguiente en postman.

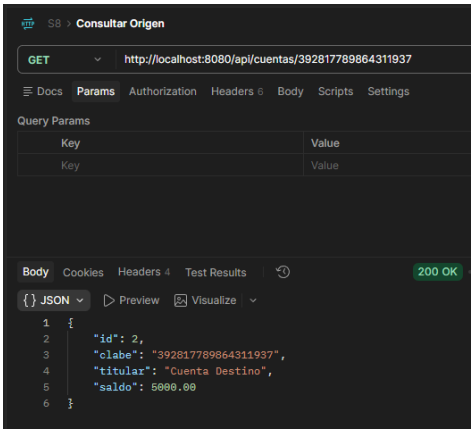


Generación de Cuenta Origen

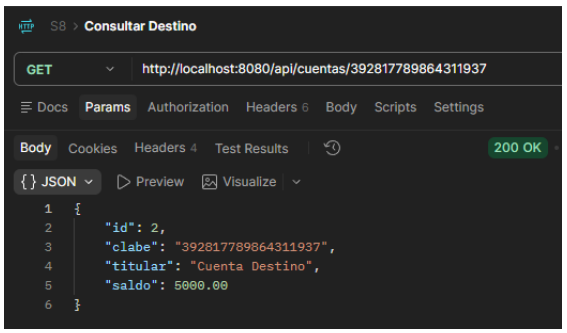


Generación de Cuenta Distinto

Ahora si ya teniendo las cuentas creadas, se realiza la consulta de saldo de cada una de las cuentas:



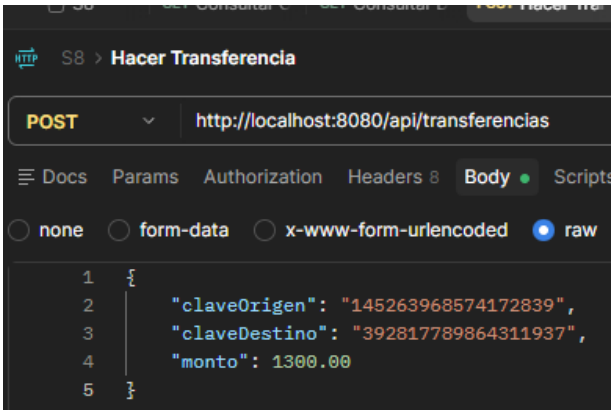
Consulta de Saldo Origen



Consulta de Saldo Distinto

Estas dos cuentas en MexiBanco serán las que nos ayuden a realizar la práctica, en este caso se utilizarán funciones utilizadas en prácticas anteriores para poder transferir y comprobar los estados de cuenta. Con esto podemos definir la problemática ¿qué pasa cuando se cae el servicio de cuentas a media transferencia?

Es por ello que se prepara el entorno para provocar esta falla, en primera instancia se genera el POST de la transferencia.

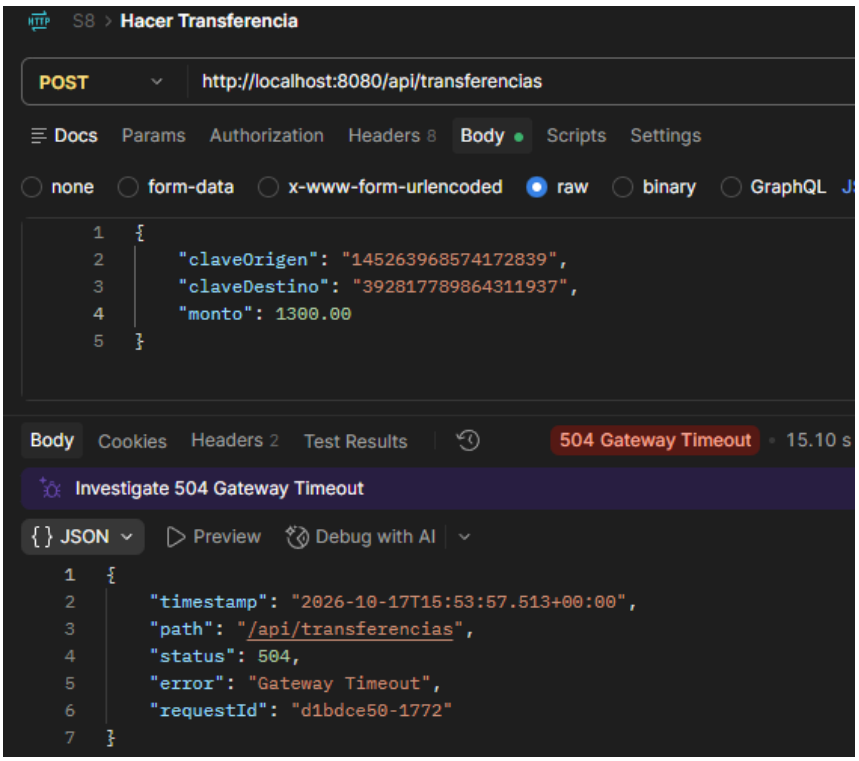


Generar Transferencia POST

Se debe cambiar el tiempo de pausa que ocurre entre la transacción, lo que detendrá el servicio de cuenta durante 20 segundos.

```
enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s08 (03-gate
way-discovery-balanceo)
$ PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia
 Container mexi-banco-v06-discovery-1 Running
 Container mexi-banco-v06-db-1 Running
 Container mexi-banco-v06-transferencia-1 Recreate
 Container mexi-banco-v06-transferencia-1 Recreated
 Container mexi-banco-v06-db-1 Waiting
 Container mexi-banco-v06-discovery-1 Waiting
 Container mexi-banco-v06-discovery-1 Healthy
 Container mexi-banco-v06-db-1 Healthy
 Container mexi-banco-v06-transferencia-1 Starting
 Container mexi-banco-v06-transferencia-1 Started
```

Ahora si lo que se tiene que hacer es mediante Postman es enviar la transferencia de un monto entre cuentas. A la misma vez que se envía, ejecutar la detención del docker con *docker compose stop cuenta* lo que mandará un error sobre el servicio de cuentas detenido, por lo que no se genera la transferencia.

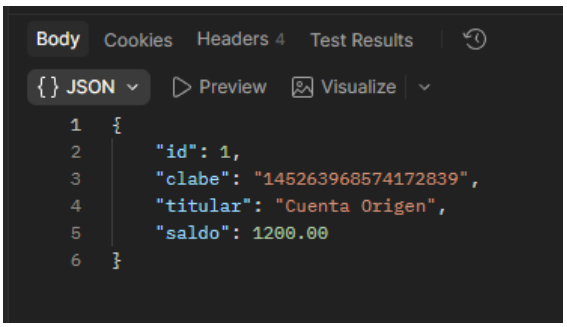


Generación de la transferencia no realizada.

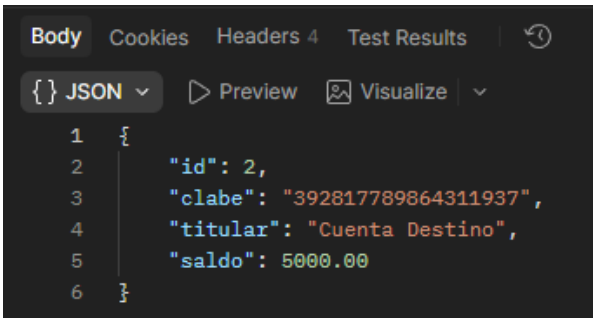
```
enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s08 (03-gate
way-discovery-balanceo)
$ docker compose stop cuenta
 Container mexi-banco-v06-cuenta-1 Stopping
 Container mexi-banco-v06-cuenta-3 Stopping
 Container mexi-banco-v06-cuenta-2 Stopping
 Container mexi-banco-v06-cuenta-3 Stopped
 Container mexi-banco-v06-cuenta-2 Stopped
 Container mexi-banco-v06-cuenta-1 Stopped
```

Detención de servicios de cuentas.

Si se vuelve a levantar el servicio de cuentas dentro del docker, lo que observamos es que la transferencia se realizó del lado de la cuenta Origen, pero el abono a la cuenta Destino no se completó, el dinero quedó en el limbo.



Cuenta Origen tiene 1300 menos.



Cuenta Destino sigue igual.

2. Tu saga.

Los pasos y la compensación de cada uno, con un diagrama de estados de la transferencia (a mano y fotografiado también vale).

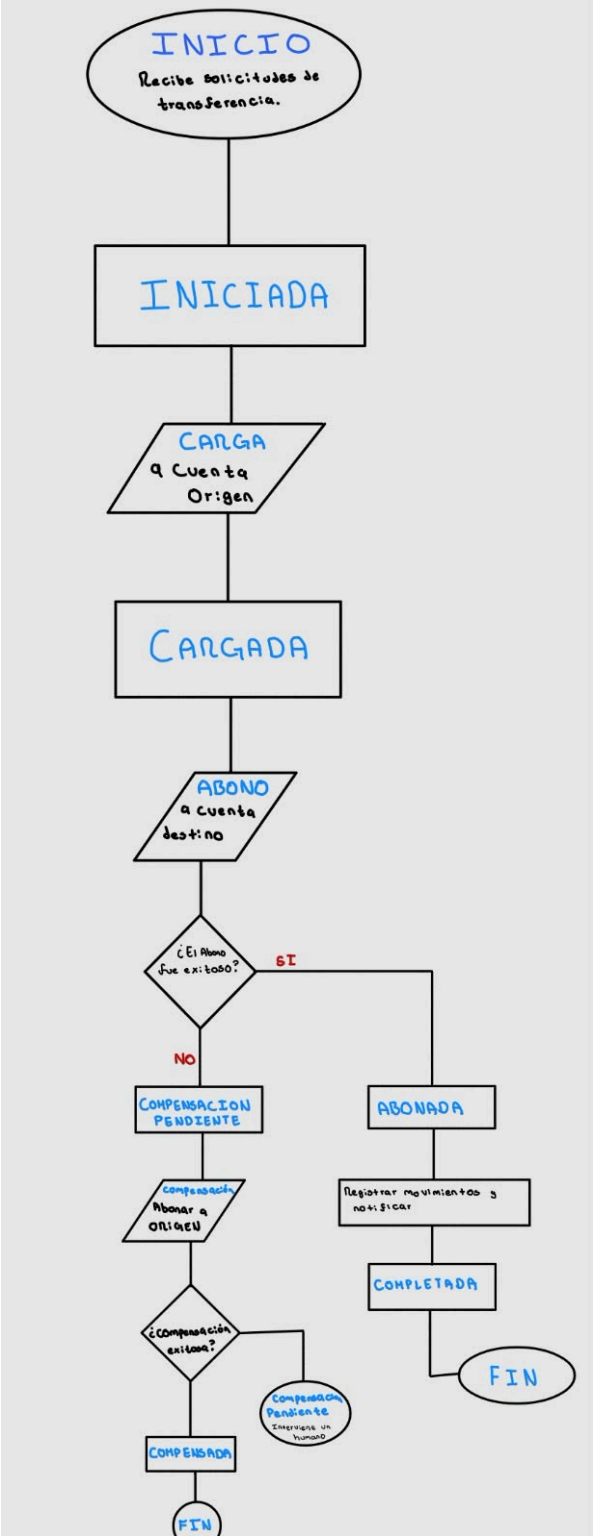
INICIADA: Aquí el orquestador recibe la solicitud de transferencia, crea el registro en la base de datos y guarda este estado lineal.

CARGADA: Se realiza el cargo a la cuenta Origen hecha por una petición HTTP, de forma que si tiene éxito, el estado avanza para reflejar que el dinero ya salió.

ABONADO: Se hace el intento por abonar el dinero en la cuenta distinto, en donde hay dos posible soluciones:

- **Flujo exitoso:** si la red responde bien y los servicios están arriba, el estado cambiaría a **ABONADA,** sigue el flujo normal y la saga termina en el estado **COMPLETADA.**
- **Flujo de Compensación:** Si el abono llegará a fallar el orquestador cambia el estado a **COMPENSACION_PENDIENTE** por seguridad del sistema. Lo que debería de hacer es activar un abono de retorno a la cuenta afectada,

para hacer la devolución del dinero. De esta manera al terminar la transferencia se indica cómo **COMPENSADA,** en dado caso que el servicio sufra una caída ese movimiento pasa a un estado de **COMPENSACION_PENDIENTE** para que al activarse el servicio de nuevo termine la saga.



3. El código clave.

Los fragmentos que cambiaste, explicados con tus palabras.

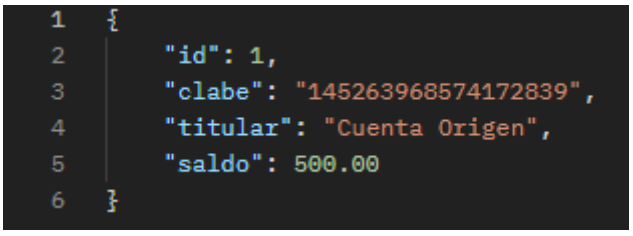
Lo primero que se modificó fue el archivo ***TransferenciaService.java***, en donde se modificaron la clase ***Transferencia*** donde se agregó un atributo de estado que se persistiera en la base de datos , en el caso del orquestador se agregó un apartado para cachar la excepción del servicio de cuentas si el destino fallaba una vez realizado el cargo. Ahora cuando eso ocurre pasa a un estado de **COMPENSACION_PENDIENTE** y manda a llamar al método compensación, la cual se encarga de mandar de regreso el monto transferido a la cuenta origen, marcando como compensada en la base de datos.

4. La compensación funciona.

La falla provocada, los saldos que regresan y el estado final en la base.



Falla provocada





Cuenta origen con el estado compensación

Para comprobar el estado de compensación se agregó una línea en el YML del docker en donde se provocaba de forma automática en vez de parar los servicios de la transferencia, donde el orquestador atrapó la excepción y ejecutó la compensación automática. En donde el saldo de la cuenta origen fue devuelto y no se quedó en el limbo.

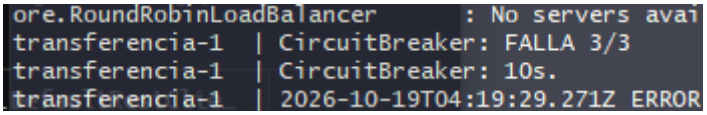
5. Cuando la compensación también falla.

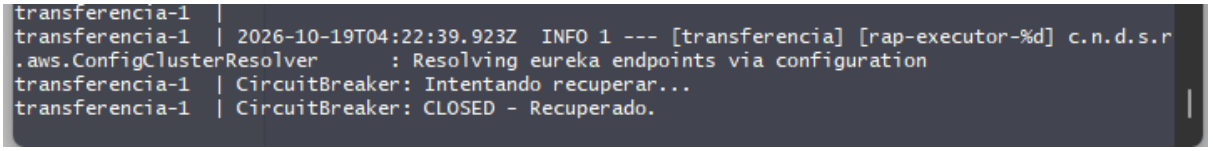
Qué estado queda y cómo lo terminarías.

En caso que el abono falle, la saga entra en modo de compensación, pero aún así si si la red se cae por completo y el abono de reserva de vuelta al origen también falla la saga no puede deshacer la operación de forma instantánea, por lo que se queda en **COMPENSACION_PENDIENTE**, de forma que para cumplir con la consistencia un worker asíncrono que esté al pendiente de checar transferencias pendientes al momento de restablecer los servicios para poder volver a reintentar la. el servicio de cuentas debe garantizar la idempotencia exigiendo un **intentoID** único.

6. El circuit breaker.

Los tres estados, con evidencia.





CLOSED: es el estado normal, las peticiones fluyen con normalidad

OPEN: Al acomular 3 fallas consecutivas, el circuito se abre y durante 10 segundos rechaza inmediatamente cualquier intento de transferencia para no saturar el servicio mientras está caído.

HALF_OPEN: Pasando los 10 segundos, el circuito permite pasar una sola petición de prueba, de manera que si esta pasa sin problema es que el servicio se recupero.

7. Lo que la saga no te da.

Un párrafo sobre el aislamiento (¿qué ve alguien que consulta el saldo de origen entre el cargo y la compensación?)

Saga no nos proporciona aislamiento como tal, en la saga los pasos se confirman de forma independiente en distintas bases de datos. Si un usuario o sistema consulta el saldo de la cuenta origen en los milisegundos que transcurren entre que se hace el cargo y ocurre la compensación por una falla, verá que su saldo disminuyó. De manera que los usuarios son capaces de ver estados intermedios temporales antes de ser revertidos por el banco.